Логика и OWL

От интеллектуального поиска к интеллектуальным агентам

Лекция 2

О. М. Атаева А. П. Халов

ВМК МГУ

Осень 2026

AI4Science AI4Research AI4Math

План лекции

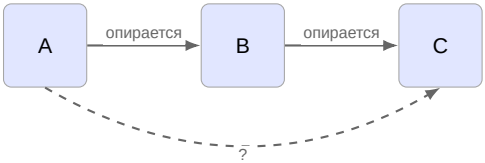
1. Задача: правило из кода переходит в данные
2. Логика: высказывания, предикаты, кванторы, отношения, правило вывода
3. От логики к OWL: разрешимые фрагменты и словарь
4. Конструкции OWL на задаче редактора
5. Вывод и непротиворечивость: правила Хорна и операции над матрицами
6. Демонстрация: положение и журнал как онтология
7. Архитектура агента: продолжение
8. Домашнее задание и итоги

Вопросы и цепочки

Вопросы по статье недели

- Hogan et al., «Knowledge Graphs», разделы 1--2.
- Отчёт о выдуманных ссылках.
- Примеры вопросов: чем граф знаний со схемой отличается от property-graph? Как записать правило, чтобы его исполняла машина? Почему отсутствие факта в графе не означает, что факт ложен?

Цепочки из ваших корпусов



Записаны два звена. Как машина получит третье?

Часть 1

Задача

Вопрос лекции

Как записать знание предметной области, чтобы машина выводила из него новые факты и обнаруживала противоречия в записи?

Шаг	Что делаем
задача	правило из кода лекции 1 переносится в данные
логика	язык для записи: высказывания, предикаты, кванторы, отношения
OWL	тот же язык в разрешимом фрагменте, словарь соответствий
конструкции	классы, свойства, ограничения на задаче редактора
вывод	правила Хорна, замыкание, проверка непротиворечивости
демонстрация	положение и журнал как онтология, машина вывода
агент	онтология в компонентах агента

Что осталось от лекции 1

Задача редактора

- Положение о статьях: 31 запись.
- Журнал рецензий: 14 строк.
- Статья P2 с тремя утверждениями.
- Вопрос: какие статьи выносить на решение и на что опирается это требование.

Что получили

- Граф и запрос пути: 14 положений для пункта 6.5.
- Четыре правила над журналом, 15 строк кода: P1, P2, P7.
- Внешний верификатор: утверждение C3 отклонено.
- Ответ с пометками источника и проверки.

Главное

Схема, структура и проверка появились вокруг модели. Правила при этом написал человек, прочитавший положение.

Правило в коде и в данных

Лекция 1: правило в коде

```
rv["своя_организация"] = (
    rv["рецензент_организация"]
    == rv["авторы_организация"])

"6.2": lambda d: not d.loc[
    d["сдана"], "своя_организация"].any()
```

Знание о требовании находится в программе.
Изменилось положение --- код переписывается.

Лекция 2: правило в данных

```
Рецензия(?r), на_статью(?r, ?s),
рецензент(?r, ?p),
из_организации(?p, ?o),
организация_авторов(?s, ?o)
  -> нарушает(?s, R62)
```

Знание о требовании записано рядом с фактами.
Его читает человек и исполняет машина вывода.

Главное

Запись знания как данных отделяет его от программы. Одна и та же машина вывода работает с любым положением.

Скелет примера



	Положение о статьях	Статья Р2
утверждение	требование 6.5 «Принятие решения»	С1: медиана к гипотенузе равна её половине
зависимость	6.5 зависит от 6.1, 6.4, термина «Решение», раздела 6	С1 опирается на определения и теорему Пифагора
обоснование	источник: текст положения	доказательство и вердикт внешней проверки

Главное

Утверждения, обоснования и зависимости между ними есть в любой предметной области. Этот скелет записывается сегодня как онтология.

Что получим к концу

	Результат	Чем получен
1	требование 6.5 зависит от 14 положений	транзитивное свойство
2	статья P3 нарушает требование 6.2	правило Хорна
3	требование 6.5 является положением и объектом	иерархия классов
4	разделы 1 и 2 базовые	определяемый класс и замыкание мира
5	строка журнала с ошибкой обнаружена	область значений и дизъюнктность

- Четыре вывода и одно обнаруженное противоречие.
- Требования 6.1 о числе рецензий и 6.6 о сроке остаются лекции 3.

Часть 2

Логика

Высказывание

Высказывание --- предложение, о котором имеет смысл спрашивать, истинно оно или ложно.

Предложение	Значение
Рецензент Rv4 из ФИЦ ИУ.	истина
У статьи P4 две сданные рецензии.	ложь
Пункт 6.5 зависит от пункта 6.4.	истина

Высказыванием не является

- вопрос: «Какие статьи готовы?»
- поручение: «Проверь журнал.»
- оценка без критерия: «Статья слабая.»
- предложение с переменной: «*x* зависит от пункта 6.4.»

Истинность определяется относительно структуры: журнала и положения. Это разобрано на лекции 1.

Связки

A	B	$A \wedge B$	$A \vee B$	$\neg A$
и	и	и	и	л
и	л	л	и	л
л	и	л	и	и
л	л	л	л	и

«и» --- конъюнкция, «или» --- дизъюнкция, «не» --- отрицание.

На журнале

- A: у статьи две сданные рецензии.
- B: есть ответ авторов.
- $A \wedge B$: условие пункта 6.5.
- $\neg A$: у статьи меньше двух сданных рецензий.

«Или» в логике неисключающее: $A \vee B$ истинно и тогда, когда истинны оба высказывания.

Импликация

A	B	$A \to B$
и	и	и
и	л	**л**
л	и	и
л	л	и

Импликация ложна в одном случае: посылка истинна, заключение ложно.

«Если запись является требованием, то у неё есть раздел»

- R65 --- требование, раздел 6 указан: правило выполнено.
- Требование без раздела: правило нарушено.
- T06 --- термин. Посылка ложна, правило о T06 ничего не утверждает и считается выполненным.

Правило говорит только о тех объектах, для которых посылка истинна.

Главное

Требования положения имеют вид «если …, то …». Импликация --- основная форма записи знания.

Равносильности

Закон	Запись	На примере
импликация через «или»	$A \to B \equiv \neg A \vee B$	запись либо не требование, либо имеет раздел
контрапозиция	$A \to B \equiv \neg B \to \neg A$	у записи нет раздела, значит она не требование
закон де Моргана	$\neg(A \wedge B) \equiv \neg A \vee \neg B$	условие 6.5 нарушено: мало рецензий или нет ответа авторов
закон де Моргана	$\neg(A \vee B) \equiv \neg A \wedge \neg B$	
двойное отрицание	$\neg\neg A \equiv A$	

- Равносильные формулы истинны в одних и тех же случаях.
- Машина вывода пользуется этими законами при преобразовании аксиом.

Предикат

Предикат --- высказывание с переменными. При подстановке объектов получается высказывание.

Предикат	Мест
Требование(x)	одно
Рецензент(x)	одно
зависит_от(x, y)	два
рецензент(r, p)	два

Подстановка

- Требование(R65) --- истина.
- Требование(T06) --- ложь.
- зависит_от(R65, R64) --- истина.
- зависит_от(R64, R65) --- ложь.

Одноместный предикат выделяет множество объектов. Двухместный предикат выделяет множество пар.

Главное

Одноместные предикаты станут классами, двухместные --- свойствами.

Кванторы

Всеобщность $\forall x$: для каждого x.
Существование $\exists x$: найдётся x.

Кванторы связывают переменную и превращают предикат в высказывание.

Порядок кванторов

- $\forall s\, \exists p$: у каждой статьи есть рецензент. У разных статей рецензенты могут быть разными.
- $\exists p\, \forall s$: есть рецензент, который рецензирует каждую статью.

Первое верно для журнала, второе нет.

«У каждой рецензии есть статья»: $\forall r\,\big(\text{Рецензия}(r) \to \exists s\,(\text{Статья}(s) \wedge \text{на_статью}(r, s))\big)$

Главное

Сочетание $\forall \ldots \to \exists$ --- самая частая форма требования: у каждого объекта одного вида есть связанный объект другого вида.

Отрицание кванторов

$$\neg \forall x\, A(x) \equiv \exists x\, \neg A(x)$$
$$\neg \exists x\, A(x) \equiv \forall x\, \neg A(x)$$

«Не у каждой статьи две рецензии» означает: найдётся статья, у которой рецензий меньше двух.

На журнале

- Чтобы опровергнуть «у каждой статьи две рецензии», достаточно предъявить P4.
- Чтобы подтвердить «нет рецензента из организации авторов», нужно просмотреть все рецензии.

Второе утверждение требует знать, что список рецензий полон. Это вопрос об открытом и закрытом мире.

Чтение формул

Фраза положения	Формула
Каждое требование относится к разделу.	$\forall x\,(\text{Требование}(x) \to \exists y\,(\text{Раздел}(y) \wedge \text{зависит_от}(x, y)))$
Рецензию выполняет рецензент.	$\forall r \forall p\,(\text{рецензент}(r, p) \to \text{Рецензент}(p))$
Раздел не является требованием.	$\forall x\,\neg(\text{Раздел}(x) \wedge \text{Требование}(x))$
У утверждения статьи есть доказательство.	$\forall u\,(\text{Утверждение}(u) \to \exists d\,(\text{Доказательство}(d) \wedge \text{доказывается}(u, d)))$

- Каждая из четырёх формул получит запись в OWL в части 4.
- Вторая формула задаёт область значений свойства, третья задаёт непересекающиеся классы.

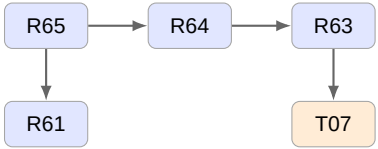
Классы и отношения

Класс --- множество объектов, для которых одноместный предикат истинен.

- Требование = {R51, …, R91}, 14 объектов.
- Рецензент = {Rv1, …, Rv6}.

Отношение --- множество пар, для которых двухместный предикат истинен.

- зависит_от = {(R65, R64), (R64, R63), …}.

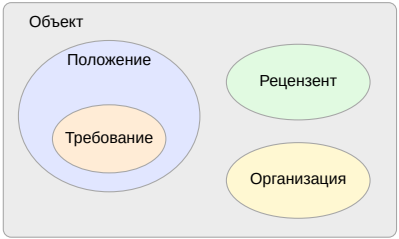


Отношение на конечном множестве изображается графом: объекты --- вершины, пары --- стрелки.

Главное

Класс и отношение --- два вида записей, из которых состоит граф знаний.

Операции над классами



Операция	Пример
включение $A \subseteq B$	Требование $\subseteq$ Положение
пересечение $A \cap B$	положения, которые являются требованиями
объединение $A \cup B$	Раздел $\cup$ Термин $\cup$ Требование $=$ Положение
дополнение $\overline{A}$	всё, что не является положением
$A \cap B = \varnothing$	Рецензент и Организация не пересекаются

Главное

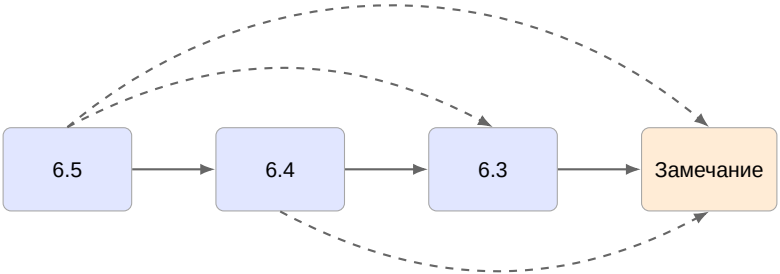
Включение классов соответствует импликации, пересечение --- конъюнкции, объединение --- дизъюнкции, дополнение --- отрицанию.

Свойства отношений

Свойство	Определение	Пример
рефлексивность	$R(x, x)$ для каждого x	«совпадает с»
симметричность	из $R(x, y)$ следует $R(y, x)$	«рецензировали одну статью»
транзитивность	из $R(x, y)$ и $R(y, z)$ следует $R(x, z)$	зависит_от
функциональность	у каждого x не больше одного y	на_статью: рецензия относится к одной статье
обратное отношение	$S(y, x)$ тогда и только тогда, когда $R(x, y)$	на_статью и имеет_рецензию

- Свойство отношения записывается один раз и действует на все пары.
- В OWL каждое из этих свойств задаётся одним словом.

Транзитивность



- Сплошные стрелки записаны в корпусе. Пунктирные следуют из транзитивности.
- Три записанных факта дают ещё три выведенных.
- Для пункта 6.5 в корпусе записаны 4 зависимости. С учётом транзитивности их 14.

Главное

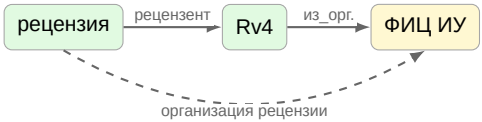
Вопрос лекции 1 «на что транзитивно опирается 6.5» получает ответ из одного свойства отношения.

Композиция отношений

Композиция $R \circ S$: пара (x, z) входит в композицию, если найдётся y с $R(x, y)$ и $S(y, z)$.

- зависит_от ∘ зависит_от: зависимости через одно звено.
- Транзитивность означает, что композиция отношения с собой содержится в нём самом.

Композиция разных отношений



Композиция «рецензент» и «из организации» даёт организацию, от которой выполнена рецензия.

Главное

Композиция превращает путь по графу в одно отношение. В части 5 она станет произведением матриц.

Правило вывода

Modus ponens

$$\frac{A \qquad A \to B}{B}$$

Над чертой посылки, под чертой заключение. Если посылки истинны, заключение истинно.

Вывод в три шага

1	зависит_от(R65, R64)	записано
2	зависит_от(R64, R63)	записано
3	зависит_от(x, y) $\wedge$ зависит_от(y, z) $\to$ зависит_от(x, z)	правило
4	зависит_от(R65, R63)	из 1, 2, 3

- Каждый шаг вывода проверяется механически.
- Выведенный факт прослеживается до записанных фактов и правила.

Главное

Вывод --- это операция над записями. Её может выполнять машина.

Часть 3

От логики к OWL

Цена выразительности

Логика первого порядка

- Предикаты любой местности, кванторы, связки.
- На ней записывается любое требование положения.
- Вопрос «следует ли формула из других» в общем случае неразрешим: процедура проверки может не завершиться.

Что это значит для задачи

- Редактору нужен ответ за конечное время.
- Машина должна завершать проверку на любой записи.
- Часть выразительности приходится отдать.

Главное

Чем больше можно записать на языке, тем труднее вычислять по записанному. Язык для записи знаний выбирается как компромисс.

Требования к языку

Требование	Что оно даёт
формальный синтаксис	запись читает программа, двух прочтений одной записи нет
формальная семантика	смысл записи определён через истинность в структуре
разрешимость	проверка следования завершается на любой записи
корректность вывода	из истинного выводится только истинное
полнота вывода	всё, что следует из записи, машина способна вывести
читаемость	запись понимает специалист предметной области

- Естественный язык выполняет последнее требование и нарушает первые пять.
- Логика первого порядка нарушает требование разрешимости.

Дескрипционные логики

Разрешимые фрагменты логики первого порядка.

- Одноместные предикаты: *понятия*, они же классы.
- Двухместные предикаты: *роли*, они же свойства.
- Константы: *индивиды*.
- Кванторы только в ограниченной форме: «существует связанный объект класса C», «все связанные объекты из класса C».

Схема и факты

TBox	Требование $\sqsubseteq$ Положение Рецензия $\sqsubseteq \exists$ на_статью.Статья
ABox	R65 : Требование зависит_от(R65, R64)

Разделение схемы и фактов введено на лекции 1.

Главное

OWL --- стандарт W3C для записи онтологий. Его логическая основа --- дескрипционная логика.

Словарь: классы

Логика	Дескрипционная логика	OWL, Manchester Syntax
$\forall x\,(A(x) \to B(x))$	$A \sqsubseteq B$	`A SubClassOf B`
$\forall x\,(A(x) \leftrightarrow B(x))$	$A \equiv B$	`A EquivalentTo B`
$A(x) \wedge B(x)$	$A \sqcap B$	`A and B`
$A(x) \vee B(x)$	$A \sqcup B$	`A or B`
$\neg A(x)$	$\neg A$	`not A`
$\neg\exists x\,(A(x) \wedge B(x))$	$A \sqcap B \sqsubseteq \bot$	`A DisjointWith B`
$\exists y\,(R(x,y) \wedge C(y))$	$\exists R.C$	`R some C`
$\forall y\,(R(x,y) \to C(y))$	$\forall R.C$	`R only C`
не меньше n значений y класса C	$\geq n\,R.C$	`R min n C`

Главное

Каждой конструкции OWL соответствует формула логики первого порядка. Смысл записи определён однозначно.

Словарь: свойства и индивиды

Логика	Что это	OWL, Manchester Syntax
$R(x,y) \wedge R(y,z) \to R(x,z)$	транзитивное свойство	`Transitive`
$R(x,y) \to R(y,x)$	симметричное свойство	`Symmetric`
$R(x,y) \wedge R(x,z) \to y = z$	функциональное свойство	`Functional`
$R(x,y) \leftrightarrow S(y,x)$	обратное свойство	`InverseOf`
$R(x,y) \to A(x)$	область определения	`Domain`
$R(x,y) \to B(y)$	область значений	`Range`
$R(x,y) \to S(x,y)$	подсвойство	`SubPropertyOf`
$A(c)$	индивид класса	`c Types A`
$R(c,d)$	факт о паре индивидов	`c Facts R d`

- Все записи левого столбца, кроме двух последних, имеют вид правила «если …, то …».

OWL 2 и профили

OWL 2 DL --- рабочий диалект лекции.

- Логическая основа: дескрипционная логика SROIQ.
- Вывод разрешим.
- В худшем случае вывод дорог, на учебных онтологиях он занимает секунды.

Профили --- фрагменты с быстрым выводом.

EL	большие иерархии классов
QL	запросы к данным в базе
RL	вывод выполняется правилами

Профиль RL близок к правилам Хорна из части 5.

Главное

Синтаксисов у OWL несколько: Manchester для чтения, Turtle для обмена, RDF/XML для хранения. Содержание записи от синтаксиса не зависит.

SROIQ и профили

Имя логики читается по буквам

	Что разрешено	В примере
$\mathcal{ALC}$	$\sqcap$, $\sqcup$, $\neg$, $\exists R.C$, $\forall R.C$	у рецензии есть статья
$\mathcal{S}$	$\mathcal{ALC}$ и транзитивные свойства	`зависит_от`
$\mathcal{R}$	иерархии и цепочки свойств	рецензент $\circ$ из организации
$\mathcal{O}$	классы из перечисленных индивидов	{МГУ} в упражнении 7
$\mathcal{I}$	обратные свойства	`имеет_рецензию`
$\mathcal{Q}$	число связей с указанием класса	не меньше двух рецензий
$(\mathcal{D})$	типы данных	оценка: число

Диалекты и их логики

Диалект	Логика	Что оставлено, вывод
OWL 2 DL	$\mathcal{SROIQ}(\mathcal{D})$	все конструкции слева. Вывод разрешим, худший случай дважды экспоненциален
OWL 2 EL	$\mathcal{EL}^{++}$	$\sqcap$, $\exists R.C$, цепочки свойств. Полиномиальный вывод
OWL 2 QL	DL-Lite$_{\mathcal{R}}$	иерархии, обратные свойства. Запрос переписывается в SQL
OWL 2 RL	DLP	аксиомы вида правил Хорна. Полиномиальный вывод

Первая версия языка: OWL DL на логике $\mathcal{SHOIN}(\mathcal{D})$, OWL Lite на логике $\mathcal{SHIF}(\mathcal{D})$.

Главное

Буквы в имени логики перечисляют разрешённые конструкции. Профиль выбирается по задаче.

Открытый мир в записи

Рецензия $\sqsubseteq \exists$ на_статью.Статья

- База данных: у рецензии нет статьи --- ошибка.
- OWL: у рецензии есть статья, она пока не названа.

Ограничение существования задаёт обязательство. Объект, выполняющий его, может быть не предъявлен.

Следствия

- Отсутствие факта не делает его ложным.
- «У статьи P4 одна рецензия» означает: записана одна, других записей нет.
- Проверка полноты данных выполняется другим средством. Это лекция 3.

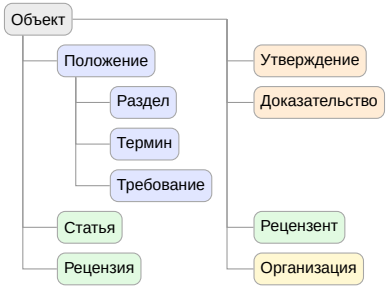
Главное

OWL выводит то, что следует из записанного. Проверять, всё ли записано, он не предназначен.

Часть 4

Конструкции OWL

Классы и иерархия



Все классы примера являются подклассами класса Объект. У класса Положение три подкласса.

Manchester Syntax

```
Class: Положение  SubClassOf: Объект
Class: Требование SubClassOf: Положение
```

Owlready2

```
with onto:
    class Объект(Thing): pass
    class Положение(Объект): pass
    class Требование(Положение): pass
    class Рецензия(Объект): pass
```

Класс онтологии записывается как класс Python. Наследование Python задаёт `SubClassOf`.

Дизъюнктность

Непересекающиеся классы не имеют общих индивидов.

- Раздел, Термин, Требование попарно не пересекаются.
- Положение, Статья, Рецензия, Рецензент, Организация, Утверждение, Доказательство попарно не пересекаются.

Без этой записи машина допускает, что один индивид является и рецензентом, и организацией.

Manchester Syntax

```
DisjointClasses:
    Раздел, Термин, Требование
```

Owlready2

```
AllDisjoint([Раздел, Термин, Требование])
AllDisjoint([Положение, Статья, Рецензия,
             Рецензент, Организация,
             Утверждение, Доказательство])
```

Главное

Дизъюнктность даёт машине вывода возможность обнаружить противоречие. Она понадобится в части 5.

Индивиды

Индивид --- конкретный объект предметной области.

Индивид	Класс
R65	Требование
T06	Термин
P3	Статья
Rv4	Рецензент
ФИЦ_ИУ	Организация

Все 31 запись корпуса и все строки журнала становятся индивидами.

Manchester Syntax

```
Individual: R65
    Types: Требование
    Annotations: rdfs:label "6.5 Принятие решения"
```

Owlready2

```
R65 = Требование("R65")
R65.label = ["6.5 Принятие решения"]
```

Разные имена не означают разных индивидов. Различие объявляется явно или следует из дизъюнктности классов.

Объектные свойства

Свойство	Откуда	Куда	Характеристика
`зависит_от`	Положение	Положение	транзитивное
`на_статью`	Рецензия	Статья	функциональное
`имеет_рецензию`	Статья	Рецензия	обратное к `на_статью`
`рецензент`	Рецензия	Рецензент	функциональное
`из_организации`	Рецензент	Организация	
`организация_авторов`	Статья	Организация	
`нарушает`	Статья	Требование	
`содержит_утверждение`	Статья	Утверждение	
`доказывается`	Утверждение	Доказательство	обратное `доказывает`

- Объектное свойство связывает два индивида.
- Свойства взяты из карты знаний лекции 1.

Области определения и значений

Область определения: класс того, от кого идёт связь.
Область значений: класс того, к кому она идёт.

Это аксиомы для вывода типа:

- из `рецензент(r, p)` следует Рецензия(*r*),
- из `рецензент(r, p)` следует Рецензент(*p*).

Manchester Syntax

```
ObjectProperty: рецензент
    Domain: Рецензия
    Range:  Рецензент
```

Owlready2

```
class рецензент(ObjectProperty):
    domain = [Рецензия]
    range  = [Рецензент]
```

Главное

Область значений не запрещает записать в поле «рецензент» организацию. Машина выведет, что эта организация является рецензентом. Ошибку обнаружит дизъюнктность.

Характеристики свойств

Характеристика	Свойство примера
транзитивное	`зависит_от`
функциональное	`на_статью`, `рецензент`
обратное	`имеет_рецензию`

Записана связь «рецензия на статью». Связь «статья имеет рецензию» машина получает сама.

Manchester Syntax

```
ObjectProperty: зависит_от
    Characteristics: Transitive
ObjectProperty: имеет_рецензию
    InverseOf: на_статью
```

Owlready2

```
class зависит_от(ObjectProperty,
                 TransitiveProperty): ...
class на_статью(ObjectProperty,
                FunctionalProperty): ...
class имеет_рецензию(ObjectProperty):
    inverse_property = на_статью
```

Главное

Одно слово `Transitive` заменяет запрос пути из лекции 1.

Свойства данных

Свойство данных связывает индивид со значением: числом, строкой, датой.

Свойство	Тип	Пример
оценка	число	4.0
шкала	целое	5
назначено	дата	2026-09-01
сдано	дата	2026-09-12
ответ_авторов	логическое	да

Owlready2

```
class оценка(DataProperty,
             FunctionalProperty):
    domain = [Рецензия]; range = [float]
class шкала(DataProperty,
            FunctionalProperty):
    domain = [Рецензия]; range = [int]

x = onto["рец_P2_Rv3"]
x.оценка, x.шкала     # 4.0, 5
```

Главное

Оценка и шкала записаны двумя свойствами одной рецензии. Первая ошибка агента из лекции 1, усреднение оценок по разным шкалам, при такой схеме видна в данных.

Ограничение существования

`R some C`: у индивида есть хотя бы одна связь *R* с индивидом класса *C*.

- У каждой рецензии есть статья.
- Каждое требование относится к разделу.
- У каждого утверждения статьи есть доказательство.

Manchester Syntax

```
Class: Рецензия
    SubClassOf: на_статью some Статья
Class: Утверждение
    SubClassOf: доказывается some Доказательство
```

Owlready2

```
Рецензия.is_a.append(на_статью.some(Статья))
Утверждение.is_a.append(
    доказывается.some(Доказательство))
```

Главное

Третье ограничение взято из программы: «у каждого утверждения есть обоснование». Это центральное звено скелета примера.

Ограничение всеобщности

`R only C`: все связи *R* индивида ведут к индивидам класса *C*.

- Доказательство доказывает только утверждения.
- Положение зависит только от положений.

Ограничение выполнено и для индивида, у которого связей *R* нет.

Manchester Syntax

```
Class: Доказательство
    SubClassOf: доказывает only Утверждение
```

`R some C`	есть хотя бы одна связь к *C*
`R only C`	нет связей к чужим классам

Главное

«Только» не означает «есть». Чтобы потребовать и то, и другое, пишут оба ограничения.

Числовое ограничение

`R min n C`, **`R max n C`**, **`R exactly n C`**.

Требование 6.1: у статьи не меньше двух рецензий.

```
Class: Статья
    SubClassOf: имеет_рецензию min 2 Рецензия

Статья.is_a.append(
    имеет_рецензию.min(2, Рецензия))
```

Что сделает машина вывода

- У статьи P4 записана одна рецензия.
- Машина заключит: у P4 есть ещё одна рецензия, она пока не названа.
- Сообщения о нарушении не будет.

Главное

В открытом мире числовое ограничение задаёт обязательство. Проверка требования 6.1 по журналу остаётся лекции 3.

Определяемый класс

`SubClassOf` задаёт необходимое условие. **`EquivalentTo`** задаёт необходимое и достаточное условие. Машина сама относит к такому классу подходящих индивидов.

Базовое положение: положение, которое ни от чего не зависит.

Manchester Syntax

```
Class: БазовоеПоложение
    EquivalentTo: Положение
        and not (зависит_от some Положение)
```

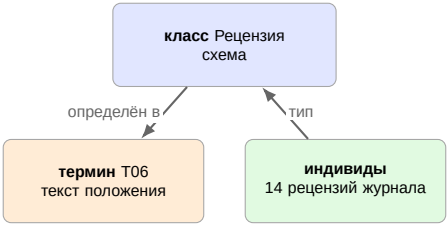
Owlready2

```
class БазовоеПоложение(Положение):
    equivalent_to = [Положение
        & Not(зависит_от.some(Положение))]
```

Главное

У раздела 1 зависимостей не записано. Машина отнесёт его к базовым только после того, как ей сообщат, что список зависимостей полон.

Термин и класс



Аннотации не участвуют в выводе. Они связывают запись с источником и дают имена для человека.

```
Class: Рецензия
    Annotations:
        rdfs:label "Рецензия"@ru,
        rdfs:isDefinedBy T06

Рецензия.isDefinedBy = [pos["T06"]]
x.label = [locstr("6.5 Принятие решения",
                  lang="ru")]
```

Слово, понятие и экземпляры из семиотического треугольника лекции 1 получают три разные записи.

Главное

Класс привязан к тексту положения. Выдуманному стандарту из ответа агента в такой записи привязаться не к чему.

Три синтаксиса

Manchester

```
Individual: R65
  Types: Требование
  Facts:
    зависит_от R61,
    зависит_от R64,
    зависит_от S06,
    зависит_от T10
```

Для чтения человеком.

Turtle

```
:R65 a :Требование ;
  rdfs:label
    "6.5 Принятие решения"@ru ;
  :зависит_от :R61, :R64,
              :S06, :T10 .
```

Для обмена. Формат лекции 4.

Owlready2

```
R65 = Требование("R65")
R65.зависит_от = [
    R61, R64, S06, T10]
```

Для работы из Python.

- **Protégé** --- редактор онтологий: иерархия классов, свойства, индивиды, запуск машины вывода.
- **Owlready2** --- библиотека Python: онтология как набор классов и объектов.
- Файл в формате RDF/XML или Turtle читают оба инструмента.

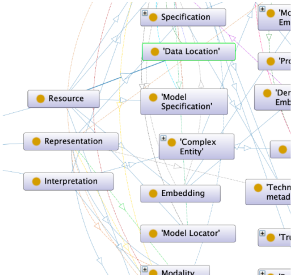
Типичные ошибки

Ошибка	Пример	Как правильно
индивид в роли подкласса	R65 записан подклассом Требования	R65 --- индивид класса Требование
`only` вместо `some`	«рецензия на_статью only Статья» в расчёте на наличие статьи	`some` требует наличия, `only` ограничивает класс
пропущена дизъюнктность	организация в поле «рецензент» не замечена	объявить классы непересекающимися
ожидание закрытого мира	`min 2` в расчёте на сообщение об ошибке	проверка данных, лекция 3
область как проверка	`Range` в расчёте на отказ принять факт	`Range` выводит тип

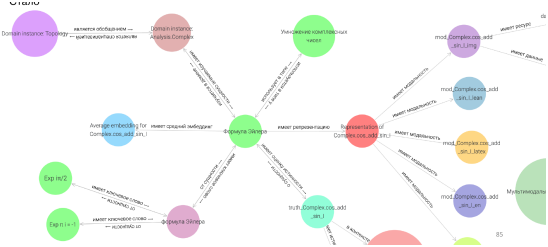
Главное

Четыре ошибки из пяти происходят от привычки к базам данных. OWL выводит следствия, запретов на ввод в нём нет.

Та же модель в SciLib



онтология области: классы и свойства



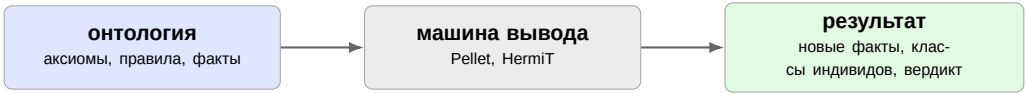
фрагмент материализованного графа знаний

- Утверждения, обоснования и зависимости между ними записаны теми же конструкциями.
- Граф знаний SciLib содержит около 66 млн троек и 724 класса OWL.
- Эту онтологию можно переиспользовать в своём проекте.

Часть 5

Вывод и непротиворечивость

Машина вывода



Задача	На примере
вывод неявных фактов	требование 6.5 зависит от раздела 1, статья P3 нарушает требование 6.2
классификация	R65 является положением, раздел 1 является базовым положением
проверка непротиворечивости	у записи есть хотя бы одна модель, противоречащих друг другу утверждений нет

Главное

Машина вывода ничего не знает о рецензировании. Смысл задают аксиомы и правила онтологии.

Правило Хорна

Правило Хорна: конъюнкция условий и одно заключение.

$$A_1 \wedge A_2 \wedge \ldots \wedge A_n \rightarrow B$$

- Условия и заключение --- предикаты с переменными.
- В заключении нет «или» и нет отрицания.
- Факт --- правило без условий.

Аксиомы примера как правила

подкласс	Требование(x) $\rightarrow$ Положение(x)
область значений	рецензент(r, p) $\rightarrow$ Рецензент(p)
транзитивность	зависит_от(x, y) $\wedge$ зависит_от(y, z) $\rightarrow$ зависит_от(x, z)

Главное

Правила Хорна дают единственный наименьший набор следствий и вычисляются за полиномиальное время. На них построены Datalog, профиль OWL 2 RL и язык правил SWRL.

Правило 6.2

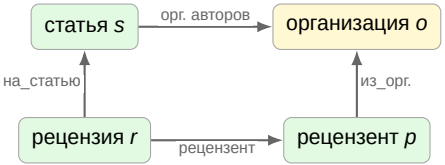
Положение	Рецензент не может состоять в организации авторов статьи.
Формула	Рецензия(r) $\wedge$ на_статью(r, s) $\wedge$ рецензент(r, p) $\wedge$ из_организации(p, o) $\wedge$ организация_авторов(s, o) $\rightarrow$ нарушает(s, R62)

SWRL в Owlready2

```
правило_62 = Imp()
правило_62.set_as_rule(
  "Рецензия(?r), на_статью(?r, ?s), "
  "рецензент(?r, ?p), из_организации(?p, ?o), "
  "организация_авторов(?s, ?o) "
  "-> нарушает(?s, R62)")
```

- Пять условий и одно заключение.
- Те же условия, что в коде лекции 1.
- Заключение ссылается на индивид R62: нарушение привязано к пункту положения.

Граница OWL



Требование 6.2 сравнивает два значения, полученные по разным путям.

Средствами классов OWL 2 DL

- Условие «одна и та же организация» цепочкой свойств точно не выражается.
- Запись без правил требует по одной аксиоме на каждую организацию. Схема начинает зависеть от данных.

Правило решает задачу одной записью.

Дальше: то же требование будет записано проверкой SHACL на лекции 3 и запросом SPARQL на лекции 4.

Главное

У каждого языка записи знаний есть граница выразительности. Правила расширяют OWL там, где нужно сравнение значений.

Замыкание по правилам

Прямой вывод

1. Найти все подстановки, при которых условия правил выполнены на фактах.
2. Добавить заключения как новые факты.
3. Повторять, пока появляются новые факты.

Состояние, в котором новых фактов нет, называется *неподвижной точкой*. Индивидов конечное число, поэтому процесс завершается.

записано	(R65, R64), (R64, R63), (R63, T07)
шаг 1	(R65, R63), (R64, T07)
шаг 2	(R65, T07)
шаг 3	новых фактов нет

На всём примере: шаг 1 даёт 73 новых факта, шаг 2 даёт 19, на шаге 3 новых фактов нет.

Главное

Замыкание по трём правилам даёт те же 14 положений для пункта 6.5 и то же нарушение 6.2, что и машина вывода.

Вывод как операция над матрицей

Отношение «подкласс» на классах записывается булевой матрицей S: строка $\sqsubseteq$ столбец.

$$S = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} \xrightarrow{S \vee S^2} S^* = \begin{pmatrix} 0 & 1 & \mathbf{1} \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}$$

Строки и столбцы: Требование, Положение, Объект.

Транзитивное замыкание $S^* = S \vee S^2 \vee S^3 \vee \ldots$ вычисляется до неподвижной точки.

Что получилось

- Записано: Требование $\sqsubseteq$ Положение, Положение $\sqsubseteq$ Объект.
- Выведено: Требование $\sqsubseteq$ Объект.
- Новая единица в матрице есть новый факт.

Главное

Вывод --- это операция над записью, её результат можно проверить вручную. Тот же приём работает для любой предметной области.

Булево произведение и композиция

Матрица D отношения `зависит_от` на цепочке R65, R64, R63, T07:

$$D = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} \quad D^2 = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$D^* = D \vee D^2 \vee D^3 = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Булево произведение

$$(D^2)_{ik} = \bigvee_j (D_{ij} \wedge D_{jk})$$

- D^2 --- зависимости через одно звено.
- D^3 --- через два звена.
- Произведение матриц есть композиция отношений из части 2.

Главное

Шаг прямого вывода по правилу транзитивности и одно булево умножение матриц --- одна и та же операция.

Распространение классов

Матрица M: строка --- индивид, столбец --- класс.
Принадлежность надклассам:

$$M' = \max\left(M,\ M \cdot S^*\right)$$

M'	Треб.	Полож.	Рец-т	Орг.	Объект
R65	1	**1**	0	0	**1**
Rv4	0	0	1	0	**1**
ФИЦ ИУ	0	0	0	1	**1**

Жирным выделены единицы, которых не было в M.

Что получилось

- R65 записан как требование.
- Выведено: R65 является положением и объектом.
- Третий вывод демонстрации получен одним умножением матриц.

Главное

Классификация индивидов по иерархии сводится к произведению матрицы принадлежности на замыкание матрицы подклассов.

Противоречие

Строка журнала с ошибкой

статья	рецензент	рец. орг.
P8	**ФИЦ ИУ**	

В поле «рецензент» записана организация.

1. Факт: рецензент(рец_P9, ФИЦ_ИУ).
2. Область значений: ФИЦ_ИУ является рецензентом.
3. Записано: ФИЦ_ИУ является организацией.
4. Рецензент и Организация не пересекаются.

Почему с этим нельзя работать

- Из противоречивой записи выводимо любое утверждение.
- Ответы машины по такой записи теряют смысл.
- Противоречие нужно найти до использования данных.

Каждое из четырёх утверждений слева по отдельности допустимо. Противоречие возникает из их сочетания.

Главное

Противоречие обнаруживается выводом: машина строит следствия и находит индивид в двух непересекающихся классах.

Проверка непротиворечивости

Матрица D задаёт пары непересекающихся классов.

$$(M' \cdot D) \odot M' > 0$$

Единица в строке означает: индивид принадлежит двум непересекающимся классам.

	Рецензент	Организация
Rv4	0	0
ФИЦ ИУ	**1**	**1**

Сообщение машины вывода

```
Explanation(s):
1) DisjointClasses(... Организация
                   ... Рецензент ...)
   рец_Р9_ошибка рецензент ФИЦ_ИУ
   рецензент range Рецензент
   ФИЦ_ИУ type Организация
```

Машина называет аксиомы и факты, из которых возникло противоречие.

Главное

Смысл задаёт онтология через аксиомы и правила. Механизм вывода и проверки одинаков для любой предметной области.

Часть 6

Демонстрация

Сценарий

Блокнот `demo_lecture2.ipynb`. Данные лекции 1: `corpus_Aprime_standards.csv`, `reviews.csv`, `paper_P2_claims.csv`.

Шаг	Что на экране	На что смотреть
1	классы и дизъюнктность	карта знаний лекции 1 переносится почти без изменений
2	свойства и характеристики	одно слово задаёт транзитивность
3	индивиды из корпуса и журнала	те же файлы, что в лекции 1
4	аксиомы и правило 6.2	правило из кода стало записью в онтологии
5	сохранение и Protégé	где видны классы, свойства, индивиды, правило
6	запуск машины вывода	четыре вывода
7	те же выводы своими руками	прямой вывод и три операции над матрицами
8	строка журнала с ошибкой	противоречие и его объяснение
9	чего в выводе нет	требования 6.1 и 6.6

Шаг 1. Классы

```
world = World()
onto = world.get_ontology(
    "http://example.org/review.owl")
with onto:
    class Объект(Thing): pass
    class Положение(Объект): pass
    class Раздел(Положение): pass
    class Термин(Положение): pass
    class Требование(Положение): pass
    class Статья(Объект): pass
    class Рецензия(Объект): pass
    class Рецензент(Объект): pass
    class Организация(Объект): pass
    class Утверждение(Объект): pass
    class Доказательство(Объект): pass
    AllDisjoint([Раздел, Термин, Требование])
    AllDisjoint([Положение, Статья, Рецензия, ...])
```

- Одиннадцать классов.
- Имена классов взяты из карты знаний лекции 1.
- Две записи о дизъюнктности.
- Имена записаны по-русски. Машина вывода работает с ними без изменений.

Шаг 2. Свойства

```
with onto:
    class зависит_от(ObjectProperty,
                     TransitiveProperty):
        domain = [Положение]; range = [Положение]
    class на_статью(ObjectProperty,
                    FunctionalProperty):
        domain = [Рецензия]; range = [Статья]
    class рецензент(ObjectProperty,
                    FunctionalProperty):
        domain = [Рецензия]; range = [Рецензент]
    class из_организации(ObjectProperty):
        domain = [Рецензент]; range = [Организация]
    class нарушает(ObjectProperty):
        domain = [Статья]; range = [Требование]
    class оценка(DataProperty, FunctionalProperty):
        domain = [Рецензия]; range = [float]
```

- Десять объектных свойств и семь свойств данных.
- Транзитивность задана словом `TransitiveProperty`.
- У каждого свойства указаны область определения и область значений.

Шаг 3. Индивиды

```
for _, r in corpus.iterrows():
    x = TYPES[r["тип"]](r["id"])
    x.label = [locstr(r["название"], lang="ru")]
    pos[r["id"]] = x
for _, r in corpus.iterrows():
    pos[r["id"]].зависит_от = [
        pos[d] for d in r["зависит_от"].split(";") if d]

for _, r in reviews.iterrows():
    x = Рецензия(f"рец_{r['статья']}_{r['рецензент']}")
    x.на_статью = art[r["статья"]]
    x.рецензент = rev[r["рецензент"]]
    x.оценка = float(r["оценка"])
    x.шкала  = int(r["шкала"])
```

Класс	Индивидов
Раздел, Термин, Требование	6, 11, 14
Статья	8
Рецензия	14
Рецензент	6
Организация	4
Утверждение, Доказательство	3, 3

У R65 записаны четыре зависимости: R61, R64, S06, T10.

Шаг 4. Аксиомы и правило

```
with onto:
    Рецензия.is_a.append(на_статью.some(Статья))
    Рецензия.is_a.append(рецензент.some(Рецензент))
    Требование.is_a.append(зависит_от.some(Раздел))
    Утверждение.is_a.append(
        доказывается.some(Доказательство))

    class БазовоеПоложение(Положение):
        equivalent_to = [Положение
            & Not(зависит_от.some(Положение))]

    правило_62 = Imp()
    правило_62.set_as_rule(
      "Рецензия(?r), на_статью(?r, ?s), "
      "рецензент(?r, ?p), из_организации(?p, ?o), "
      "организация_авторов(?s, ?o) "
      "-> нарушает(?s, R62)")
```

- Четыре ограничения существования.
- Один определяемый класс.
- Одно правило.

В лекции 1 требование 6.2 было строкой программы. Здесь оно записано в онтологии и сохраняется в файл вместе с фактами.

Шаг 5. Файл и Protégé

```
onto.save(file="data/review.owl",
          format="rdfxml")
# 565 троек

:R65 a :Требование, owl:NamedIndividual ;
    rdfs:label "6.5 Принятие решения"@ru ;
    :зависит_от :R61, :R64, :S06, :T10 ;
    :источник "Положение о статьях §6.5" .
```

Файл `review.owl` открывается в Protégé. Файл `review.ttl` удобен для чтения.

Что смотреть в Protégé

Вкладка	Что видно
Classes	иерархия и дизъюнктность
Object properties	области и характеристики
Individuals	R65, P3, рецензии
SWRL	правило 6.2
Reasoner	запуск машины вывода

Главное

Онтология --- это файл. Его читают редактор, библиотека Python и машина вывода.

Шаг 6. Машина вывода

```
sync_reasoner_pellet(world,
    infer_property_values=True)

1. R65 зависит_от: 14 положений
   R61, R63, R64, S01, S03, S06, T01, T04,
   T05, T06, T07, T08, T09, T10
2. нарушают R62: P3
3. типы R65: Объект, Требование, Положение
4. БазовоеПоложение до замыкания: []

close_world(pos["S01"], Properties=[зависит_от])
close_world(pos["S02"], Properties=[зависит_от])
4. БазовоеПоложение после замыкания: S01, S02
```

- Записано четыре зависимости, выведено четырнадцать. То же число дал запрос пути в лекции 1.
- Нарушение 6.2 найдено правилом.
- Четвёртый вывод пуст до тех пор, пока машине не сообщено, что список зависимостей у разделов 1 и 2 полон.

Главное

Три вывода получены без единой строки кода с правилами. Четвёртый показывает открытый мир в действии.

Шаг 7. Своими руками

```
RULES = [
 ("транзитивность",
  [("зависит_от","?x","?y"), ("зависит_от","?y","?z")],
  ("зависит_от","?x","?z")),
 ("требование 6.2", [...], ("нарушает","?s","R62")),
 ("область значений",
  [("рецензент","?r","?p")], ("тип","?p","Рецензент")),
]
closed = closure(facts, RULES)

шаг 1: новых фактов 73
шаг 2: новых фактов 19
шаг 3: новых фактов 0
R65 зависит_от: 14 положений
нарушают R62: ['P3']
```

Прямой вывод: 20 строк Python над тройками. Машина вывода и Java для него не нужны.

Три операции NumPy

- S^*: замыкание иерархии классов.
- $M' = \max(M, M \cdot S^*)$: R65 получает классы Положение и Объект.
- $(M' \cdot D) \odot M'$: проверка непротиворечивости.

Главное

Результаты совпадают с выводом машины. Механизм вывода можно проверить вручную.

Шаг 8. Строка с ошибкой

```
ошибка = Рецензия("рец_P9_ошибка")
ошибка.на_статью = art["P8"]
ошибка.рецензент = org["ФИЦ ИУ"]   # организация!

непротиворечива: False
Explanation(s):
1) DisjointClasses(Доказательство Организация
       Положение Рецензент Рецензия Статья ...)
   рец_P9_ошибка рецензент ФИЦ_ИУ
   рецензент range Рецензент
   ФИЦ_ИУ type Организация

после исправления строки, непротиворечива: True
```

Матричная проверка

```
до ошибки: противоречий нет
после ошибки:
  [('ФИЦ ИУ', ['Рецензент', 'Организация'])]
```

- Машина называет четыре участника противоречия.
- Матричная проверка называет индивид и пару классов.
- После удаления строки запись снова непротиворечива.

Шаг 9. Чего в выводе нет

статья	вывод машины	сдано	макс. дней
P1	---	2	17
P2	---	2	19
P3	нарушает R62	2	17
P4	---	**1**	12
P5	---	2	**27**
P6	---	2	13
P7	---	2	16
P8	---	**0**	---

- P4 и P8 нарушают требование 6.1, P5 нарушает требование 6.6. Машина вывода об этом не сообщает.
- Отсутствие записи о второй рецензии в открытом мире не означает, что её нет.
- Сравнение дат относится к проверке данных.

Обе проверки выполняются на замкнутом мире средствами SHACL. Это лекция 3.

Главное

Машина вывода отвечает на вопрос «что следует из записанного». Вопрос «всё ли записано и верны ли значения» решает проверка данных.

Часть 7

Архитектура агента: продолжение

Агент после лекции 1

	Компонент	В лекции 1
p_θ	модель	любая, сменная
D	ведущее приложение	цикл вызовов
T	инструменты	поиск, граф, правила, верификатор
P	контекст и инструкции	корпус, `AGENTS.md`
G	цель	список статей с причинами
V	верификатор	правила над журналом

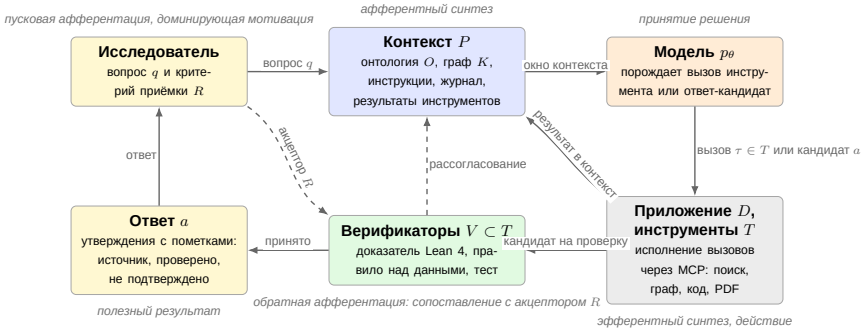


Мини-агент демонстрации вызывал четыре инструмента. Правило 6.2 находилось в коде инструмента `check_rules`.

Главное

Знание о требованиях положения находилось внутри инструмента. Агент видел только результат проверки.

Функциональная система



Функциональная система П. К. Анохина: компоненты объединены ради заранее заданного результата. Критерий приёмки *R* играет роль акцептора результата. Внешняя проверка сопоставляет результат с критерием. Рассогласование возвращает систему к новому циклу.

Онтология в компонентах агента

Компонент	Что добавляет онтология
контекст P	схема области входит в контекст: словарь классов и свойств в `AGENTS.md`, типы сущностей в ответах инструментов
инструменты T	аргументы инструментов типизируются классами онтологии. Правило 6.2 переходит из кода инструмента в данные
верификатор V	машина вывода выводит факты и проверяет непротиворечивость записи. Противоречие возвращается агенту как рассогласование
цель G	критерий приёмки записывается аксиомой или правилом. Акцептор результата получает формальную запись
модель p_θ	без изменений. Замена модели не затрагивает схему и проверку

Главное

Онтология делает явной ту часть контекста и цели, которая находилась в коде и в голове автора правил.

Машина вывода как инструмент

```
infer(article)
  -> {"выведено": [{"факт": "P3 нарушает R62 ...",
                    "правило": "правило 6.2 (SWRL)"}]}

graph_deps(doc_id)
  -> {"зависит_от": [...],
      "основание": "транзитивность зависит_от"}

check_consistency()
  -> {"непротиворечива": true | false,
      "объяснение": "..."}
```

Пометки ответа

источник	путь к записи корпуса
проверено	внешний верификатор
выведено	правило или аксиома
противоречие в данных	сообщение машины вывода
не подтверждено	сведений нет

Главное

Каждый выведенный факт возвращается вместе с правилом, по которому он получен. Это второй уровень объяснения из лекции 1.

Прогон: вывод

Вопрос: нарушает ли статья P3 требования положения и на какие положения опирается нарушенное требование?

```
[1] модель вызвала check_consistency({})
    <- {"непротиворечива": true}
[2] модель вызвала infer({'article': 'P3'})
    <- {"статья": "P3", "выведено": [{"факт": "P3 нарушает R62 (6.2 Независимость рецензента)",
                                      "правило": "правило 6.2 (SWRL)"}], ...}
[3] модель вызвала graph_deps({'doc_id': 'R62'})
    <- {"положение": "R62", "зависит_от": ["S01 Раздел 1. Область применения", "S03 ...", "S06 ...", ...]}
[4] ответ модели:
Запись непротиворечива [найдено: check_consistency]. Статья P3 нарушает требование 6.2
«Независимость рецензента» [выведено: правило 6.2]. Требование 6.2 опирается на раздел 6, раздел 3,
раздел 1 и термины Рецензент, Автор, Статья, Редакционная коллегия [найдено: graph_deps].
О требованиях 6.1 и 6.6 инструменты сведений не дали [не подтверждено].
```

Вызовы инструментов в блокноте настоящие. Без ключа доступа к модели текст её ответа берётся из записи.

Прогон: противоречие

Тот же вопрос. В журнале строка с ошибкой.

```
[1] модель вызвала check_consistency({})
    <- {"непротиворечива": false,
        "объяснение": "DisjointClasses(... Организация ... Рецензент ...)
                      рец_P9_ошибка рецензент ФИЦ_ИУ ..."}
[2] ответ модели:
Запись журнала противоречива [противоречие в данных]: индивид ФИЦ_ИУ указан рецензентом
в строке рец_P9_ошибка и одновременно является организацией.
Выводы о статьях не делаю до исправления строки.
```

- Инструкция агента требует проверить непротиворечивость до любых выводов.
- Противоречие возвращается как рассогласование. Агент останавливается и сообщает причину.
- В лекции 1 такая строка прошла бы незамеченной.

Главное

Проверка непротиворечивости защищает агента от выводов по испорченным данным. На лекциях 9--11 этот агент получит память и интерфейс MCP.

Часть 8

Домашнее задание

Что сдать

Задание лекции 2 --- вторая часть задания модуля 1. Оно выполняется на корпусе и карте знаний лекции 1.

Мини-практика: повторить блокнот лекции, открыть `review.owl` в Protégé, запустить машину вывода, добавить строку с ошибкой и исправить её.

Артефакт	Содержание	Критерий приёмки
`kg/ontology.ttl`	не меньше 6 классов с иерархией, 3 свойств, 2 ограничений, дизъюнктность для двух классов, 5--8 индивидов из корпуса	открывается в Protégé, машина вывода не находит противоречий
`kg/ONTOLOGY.md`	описание модели, 2--3 высказывания с записью в OWL, один выведенный факт, разбор одного противоречия	высказывания соответствуют аксиомам
`diary.md` `AGENTS.md`	запись о работе словарь классов в разделе «чего агент не знает сам»	заполнена дополнен

Образец: описание модели

kg/ONTOLOGY.md

```
# Онтология: положение о статьях

## Классы
Положение > Раздел, Термин, Требование
Статья, Рецензия, Рецензент, Организация

## Свойства
зависит_от: Положение -> Положение, транзитивное
рецензент:  Рецензия -> Рецензент, функциональное

## Высказывания
1. У каждой рецензии есть статья.
   Рецензия SubClassOf на_статью some Статья
2. Зависимость положений транзитивна.
   зависит_от Characteristics: Transitive
```

- Классы и свойства перечислены с иерархией и областями.
- Каждое высказывание дано дважды: фразой и записью в OWL.
- По описанию модель читается без открытия файла онтологии.

Карта знаний из задания лекции 1 служит черновиком этого описания.

Образец: вывод и противоречие

Продолжение `kg/ONTOLOGY.md`

```
## Выведенный факт
Записано: R65 зависит_от R64, R64 зависит_от R63.
Выведено: R65 зависит_от R63.
Основание: транзитивность зависит_от.

## Противоречие
Внесено: в поле «рецензент» записана
  организация ФИЦ_ИУ.
Сообщение машины вывода:
  рецензент range Рецензент
  ФИЦ_ИУ type Организация
  DisjointClasses(Рецензент, Организация)
Исправлено: строка удалена,
  запись непротиворечива.
```

Выведенный факт

- Что записано, что выведено, по какому основанию.

Противоречие

- Что внесено намеренно.
- Что сообщила машина вывода.
- Как исправлено.

Обе записи помечаются как проверенные запуском машины вывода.

Критерии и типичные ошибки

Содержательные критерии

- Подкласс является частным случаем надкласса.
- Индивиды не стоят на месте классов.
- Есть ограничение и свойство с характеристикой.
- Машина вывода получает хотя бы один незаписанный факт.
- Запись непротиворечива.

Формальные критерии проверяет `scripts/check_l2.py`. Рубрика модульного задания: корректность 40 %, декомпозиция 25 %, инструменты 20 %, воспроизводимость 15 %.

Типичные ошибки

- Все записи корпуса сделаны классами, индивидов нет.
- Иерархия плоская: все классы на одном уровне.
- Свойства без областей определения и значений.
- Дизъюнктность не объявлена, противоречие получить нельзя.
- Ожидание сообщения об ошибке от числового ограничения.
- Выведенный факт совпадает с записанным.

Упражнения

1. Перевести шесть фраз положения в аксиомы Manchester Syntax.
2. Для пяти аксиом определить вид ограничения: существование, всеобщность, число.
3. Найти три ошибки в заготовке онтологии.
4. Выписать вручную замыкание по правилу транзитивности для пункта 9.1.
5. Вычислить S^{*} и M' для иерархии из четырёх классов.
6. Найти противоречие в заготовке из шести фактов матричной проверкой.
7. Записать требование 6.2 аксиомами OWL для двух организаций, без правила.

Ответы даны в конце файла упражнений. Памятка по Protégé выдаётся вместе с упражнениями.

Главное

Упражнения служат для закрепления и не оцениваются. Статья недели: Krötzsch, Simančík, Horrocks, «A Description Logic Primer», разделы 1--3.

Итог лекции 2

1. Высказывания, предикаты и кванторы дают язык для записи знания предметной области.
2. Каждой конструкции логики соответствует конструкция OWL. Смысл записи определён однозначно.
3. Онтология разделяет схему и факты. Классы, свойства и ограничения образуют схему, индивиды и связи между ними образуют факты.
4. Замыкание по правилам Хорна даёт факты, которых никто не записывал: 14 положений для пункта 6.5, нарушение требования 6.2 статьёй P3.
5. Вывод, распространение классов и проверка непротиворечивости выражаются операциями над булевыми матрицами.
6. Противоречивая запись обнаруживается до использования: строка журнала с организацией в поле «рецензент».
7. Онтология входит в контекст агента. Машина вывода становится его инструментом и верификатором.

Главное

Знание о требованиях положения перенесено из программы в данные. Вывод и проверку согласованности выполняет машина.

К следующему занятию

Лекция 3. Проектирование онтологии и проверка данных. Онтология проектируется от вопросов исследователя. Требования 6.1 и 6.6, оставшиеся без проверки, записываются средствами SHACL и проверяются на замкнутом мире.

- **Статья недели:** Krötzsch, Simančík, Horrocks, «A Description Logic Primer», разделы 1--3. Вопрос: чем ограничение существования отличается от ограничения всеобщности на примере из своей области.
- **Принести:** три-пять вопросов исследователя из задания лекции 1. Они станут компетентностными вопросами к онтологии.
- **Установить:** библиотеку pySHACL.
- **Сделать:** найти в своей области одно требование, которое проверяется подсчётом или сравнением значений.

Литература I

1. Тарский А. Введение в логику и методологию дедуктивных наук. Главы II, IV, V.
2. Верещагин Н. К., Шень А. Языки и исчисления. МЦНМО. Главы 1--2.
3. Krötzsch M., Simančík F., Horrocks I. A Description Logic Primer. arXiv:1201.4089, 2012.
4. Baader F. et al. (eds.). The Description Logic Handbook. Cambridge University Press, 2003. Главы 1--2.
5. W3C. OWL 2 Web Ontology Language Primer. 2nd ed., 2012.
6. W3C. OWL 2 Web Ontology Language Profiles. 2nd ed., 2012. Профиль OWL 2 RL.
7. Horrocks I., Kutz O., Sattler U. The Even More Irresistible SROIQ. KR 2006.
8. Horrocks I. et al. SWRL: A Semantic Web Rule Language Combining OWL and RuleML. W3C Member Submission, 2004.
9. Motik B., Sattler U., Studer R. Query Answering for OWL-DL with Rules. Journal of Web Semantics, 3(1), 2005.
10. Russell S., Norvig P. Artificial Intelligence: A Modern Approach. Главы 7 и 9: прямой вывод по правилам Хорна.
11. Dowling W. F., Gallier J. H. Linear-time algorithms for testing the satisfiability of propositional Horn formulae. Journal of Logic Programming, 1(3), 1984.

Литература II

12. Warshall S. A Theorem on Boolean Matrices. Journal of the ACM, 9(1), 1962.
13. Rector A. et al. OWL Pizzas: Practical Experience of Teaching OWL-DL: Common Errors and Common Patterns. EKAW 2004, LNCS 3257.
14. Horridge M. A Practical Guide to Building OWL Ontologies Using Protégé 4 and CO-ODE Tools. 2011.
15. Lamy J.-B. Ontologies with Python. Apress, 2021. Главы 3, 5--7.
16. Lamy J.-B. Owlready: Ontology-oriented programming in Python with automatic classification and high level constructs for biomedical ontologies. Artificial Intelligence in Medicine, 80, 2017.
17. Sirin E. et al. Pellet: A practical OWL-DL reasoner. Journal of Web Semantics, 5(2), 2007.
18. Glimm B. et al. HermiT: An OWL 2 Reasoner. Journal of Automated Reasoning, 53(3), 2014.
19. Hogan A. et al. Knowledge Graphs. ACM Computing Surveys, 54(4), 2021.
20. Анохин П. К. Очерки по физиологии функциональных систем. М.: Медицина, 1975.
21. Yao S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.
22. W3C. Shapes Constraint Language (SHACL). W3C Recommendation, 2017. Для лекции 3.
23. Baader F., Brandt S., Lutz C. Pushing the $\mathcal{EL}$ Envelope. IJCAI 2005. Логика профиля OWL 2 EL.
24. Calvanese D. et al. Tractable Reasoning and Efficient Query Answering in Description Logics: The DL-Lite Family. Journal of Automated Reasoning, 39(3), 2007. Логика профиля OWL 2 QL.
25. Grosof B. N., Horrocks I., Volz R., Decker S. Description Logic Programs: Combining Logic Programs with Description Logic. WWW 2003. Основа профиля OWL 2 RL.

Приложение A. Шпаргалка

Что хотим сказать	Manchester Syntax	Owlready2
A --- частный случай *B*	`A SubClassOf B`	`class A(B): pass`
A и *B* не пересекаются	`A DisjointWith B`	`AllDisjoint([A, B])`
c --- объект класса *A*	`c Types A`	`c = A("c")`
свойство от *A* к *B*	`Domain: A, Range: B`	`domain = [A]; range = [B]`
свойство транзитивно	`Characteristics: Transitive`	`class R(ObjectProperty, TransitiveProperty)`
S обратно к *R*	`InverseOf: R`	`inverse_property = R`
у каждого *A* есть связь *R* с *C*	`A SubClassOf R some C`	`A.is_a.append(R.some(C))`
все связи *R* ведут в *C*	`A SubClassOf R only C`	`A.is_a.append(R.only(C))`
не меньше *n* связей	`R min n C`	`R.min(n, C)`
A определён условием	`A EquivalentTo ...`	`equivalent_to = [...]`
правило	вкладка SWRL	`Imp().set_as_rule("... -> ...")`
запуск машины вывода	меню Reasoner	`sync_reasoner_pellet(infer_property_values=True)`

Три операции над матрицами: $S^* = S \vee S^2 \vee \ldots$, $M' = \max(M, M \cdot S^*)$, $(M' \cdot D) \odot M' > 0$.

Приложение Б. Словарь лекции

Термин	Значение	Термин	Значение
высказывание	предложение с истинностным значением	индивид	конкретный объект области
предикат	высказывание с переменными	объектное свойство	связь двух индивидов
квантор	«для каждого», «существует»	свойство данных	связь индивида со значением
импликация	«если …, то …»	область определения, значений	аксиомы вывода типа по связи
класс	множество объектов предиката	ограничение	условие на связи индивидов класса
отношение	множество пар	определяемый класс	класс, заданный условием
транзитивность	из $R(x, y)$ и $R(y, z)$ следует $R(x, z)$	дизъюнктность	отсутствие общих индивидов
композиция	путь из двух отношений	правило Хорна	условия и одно заключение
дескрипционная логика	разрешимый фрагмент логики первого порядка	замыкание	добавление следствий до неподвижной точки
TBox, ABox	схема, факты	непротиворечивость	существование модели записи
OWL 2 DL	диалект OWL с разрешимым выводом	машина вывода	программа, строящая следствия
SWRL	язык правил для OWL	Protégé, Owlready2	редактор и библиотека Python

Приложение B. Вопросы для самопроверки

1. В каком единственном случае импликация ложна? Что это значит для требования «если запись является требованием, то у неё есть раздел»?
2. Чем $\forall s\, \exists p$ отличается от $\exists p\, \forall s$ на примере статей и рецензентов?
3. Какая конструкция OWL соответствует формуле $\forall x\, (A(x) \to B(x))$?
4. Чем `R some C` отличается от `R only C`?
5. Что выведет машина из факта `рецензент(r, p)` и области значений свойства?
6. Почему числовое ограничение `min 2` не сообщает о статье с одной рецензией?
7. Что такое правило Хорна? Запишите правило для требования 6.2.
8. Что такое неподвижная точка замыкания и почему процесс завершается?
9. Что означает единица в матрице S^*, которой не было в S?
10. Какие четыре утверждения участвуют в противоречии со строкой «ФИЦ ИУ»?
11. Почему требование 6.2 не выражается одними классами OWL 2 DL?
12. Что онтология добавляет в контекст, инструменты и верификатор агента?

Приложение Г. Вариант по математике

Корпус `corpus_A_planimetry.csv`: 30 утверждений планиметрии. Статья P2: утверждения C1--C3 с доказательствами. Скелет тот же: утверждение, обоснование, зависимость.

классы	Утверждение, Теорема, Лемма, Определение, Аксиома, Доказательство
иерархия	Лемма $\sqsubseteq$ Утверждение $\sqsubseteq$ Объект
свойства	`зависит_от`, `доказывается`
свойство данных	`вердикт` внешней проверки

Что показывает блокнот, шаг 11

- Замыкание сходится за четыре шага.
- Теорема Пифагора (A14) зависит от 11 утверждений, как в лекции 1.
- Утверждение C1 статьи P2 зависит от 13 утверждений корпуса.
- Записано: Лемма $\sqsubseteq$ Утверждение, Утверждение $\sqsubseteq$ Объект. Выведено: Лемма $\sqsubseteq$ Объект.
- Вердикты: C1 и C2 приняты, C3 отклонено.

Вопросы

oataeva@frccsc.ru khalov.a@phystech.edu